

DocuRAG

How the features work, how they connect, and what to say in an interview.

DocuRAG is a document question-answering app. You upload a PDF or a text file. The app cuts it into short pieces, stores a meaning-fingerprint for each piece, and answers later questions from the pieces that match. A normal chatbot answers from memory and can sound sure while being wrong. This app looks in your file first.

How the pieces connect

Two paths share one index. The upload path writes the index. The question path reads it. They must use the same embedding model, or the search compares numbers that do not live in the same space.

Upload path

1. Browser sends PDF or TXT to FastAPI.
2. pypdf or a text decode extracts the words. PDF pages are kept.
3. The splitter makes ~1,000-character chunks with ~150 characters of overlap.
4. Gemini embeddings turn each chunk into a vector.
5. Chroma stores vector, text, filename, chunk index, and page in that session's collection.

Question path

1. The same embedding model turns the question into a vector.
2. Chroma returns the nearest chunks in that session only, up to 8.
3. Those excerpts are stuffed into one prompt.
4. Gemini 3.8 Flash writes a fact, a labeled inference, or a refusal.
5. The UI shows the answer. Sources stay collapsed until opened.

Stack: React and Vite and Tailwind on the left, FastAPI and LangChain on the right, ChromaDB on disk, Gemini for embeddings and for the written answer. Docker Compose runs the API on port 8000 and nginx serving the built UI on port 8080. Locally the UI is `http://127.0.0.1:5173` and the API is `http://127.0.0.1:8000`.

Features and how each one works

Upload PDF or TXT

The left panel accepts drag-and-drop or a file picker, up to 10 files and 20 MB each. FastAPI receives them as multipart form data. A bad type, an empty file, or a PDF with no text layer is rejected before anything is embedded.

Extract text, and keep the page

pypdf reads each PDF page on its own, so a later chunk still knows it came from page 2. A TXT file is decoded as UTF-8, then Latin-1. Windows line endings are normalized so paragraph breaks are real newlines.

Split into overlapping chunks

LangChain RecursiveCharacterTextSplitter cuts the text into pieces of about 1,000 characters, with about 150 characters repeated on the next chunk. The cut is on spaces, so a word is not sliced in half. Overlap exists so a fact sitting on the boundary still appears whole in one chunk.

Embed each chunk

Gemini embedding model gemini-embedding-001 turns each chunk into a vector of 3,072 numbers. Similar meaning lands near similar meaning, even when the wording differs. Document chunks use the RETRIEVAL_DOCUMENT task; the later question uses RETRIEVAL_QUERY.

Store one collection per session

ChromaDB on local disk holds the vector, the chunk text, the filename, the chunk index, and the page. Each browser session gets its own collection. Search cannot see another session's files, and Clear session is one collection delete. Data survives an API restart.

Ask in normal language

The question is embedded with the same model. Chroma returns the nearest chunks under cosine distance, up to 8. A short file such as a resume is included whole, so page 1 skills and page 2 projects can be used together.

Three kinds of answer

If the file states the fact, Gemini says it and names the file. If you ask for a judgment the file never writes down, such as a salary, the answer starts with "This is not stated in the document" and estimates only from roles, skills, dates, and locations that are in the excerpts. If the file is unrelated, the reply is exactly "I don't have enough information."

Collapsed sources

Under the answer, a one-line Sources label shows how many excerpts were retrieved. Opening it lists filename, page, chunk index, and a short snippet, in rank order. Those citations are the chunks that were retrieved, not a citation string the model invented.

Session survives refresh

The React app stores the session id and the chat transcript in localStorage. The document list is loaded again from the API. The vectors themselves stay on the server.

The key stays on the server

GEMINI_API_KEY is read from the environment or from a gitignored .env file. The browser never receives it. The default chat model is gemini-3.8-flash, because new API keys cannot call gemini-2.5-flash.

What talks to what

The React app never opens Chroma and never calls Gemini. It only calls four HTTP routes. FastAPI calls the rag package. ingest.py extracts, chunks, and stores. retrieve.py searches and builds the prompt. vectorstore.py owns the Chroma client. llm.py builds the Gemini embedding client and the chat client.

Route	Who calls it	What it does
POST /documents/upload	Upload panel	Index files. Returns session id and chunk counts. Optional session id appends.
GET /documents/{session_id}	App on refresh	Lists filenames and how many chunks each file became.
DELETE /documents/{session_id}	Clear session	Deletes that Chroma collection.
POST /chat/{session_id}	Chat panel	Returns the answer and the retrieved sources.

A request that mixes safe and unsafe work stops at the boundary of the file. Empty files, wrong types, and Gemini failures never write a partial index: the new chunk ids are deleted, and a brand-new session collection is dropped if the embed fails.

Interview questions and answers

Say these in your own words. The point of each answer is the why, not the class name.

1. What is RAG, in one minute?

Retrieval-Augmented Generation answers a question in two steps. First retrieve the passages in the user's documents that are about the question. Then generate the answer from those passages instead of from the model's memory alone. The model is still the writer. The documents are the evidence. That is why a chatbot stops inventing a vacation policy that was never in the file.

2. Why not paste the whole PDF into the prompt?

A long file wastes tokens, costs more, and the model pays less attention to the one relevant paragraph. Chunking plus search sends only the passages that match the question. For a short resume DocuRAG raises the cap to 8 chunks so the whole file still fits. For a large library the cap keeps the prompt focused.

3. Walk through an upload.

The browser posts the files to POST /documents/upload. The backend checks type and size, extracts text, splits it, and calls Chroma add_documents. Chroma calls the Gemini embedding model and stores each vector with filename, chunk index, and page. The response is a session id and the chunk counts. A later upload with the same session id appends to that collection.

4. Why are chunks 1,000 characters with 150 of overlap?

1,000 characters is about 200 to 250 tokens: one or two paragraphs, specific enough to point at a fact and long enough to keep the sentence around it. Overlap repeats the tail of one chunk at the head of the next so a sentence on the cut is not lost. LangChain applies overlap to the pieces it split, not to raw characters. A whole paragraph is longer than 150 characters, so splitting on blank lines would copy nothing. DocuRAG splits on spaces. Each piece is a word, about 150 characters are repeated, and words stay intact.

5. What is an embedding?

An embedding is a list of numbers that represents meaning. Two sentences about vacation days land near each other even if one says PTO and the other says 18 vacation days. Chroma then finds nearest neighbors. The same embedding model must be used when indexing and when searching. Vectors from two models are not comparable, so changing GEMINI_EMBED_MODEL means deleting the index and uploading again.

6. Why RETRIEVAL_DOCUMENT and RETRIEVAL_QUERY?

gemini-embedding-001 can tag a text as a document to store or as a question to search. LangChain does that automatically: embed_documents uses RETRIEVAL_DOCUMENT and embed_query uses RETRIEVAL_QUERY. The two vectors are trained to match each other in a search, which is better than embedding both sides as generic text.

7. Why one Chroma collection per session?

Search is automatically limited to that user's files, and delete is one drop. The alternative is one shared collection plus a metadata filter on session id. That also works, but forgetting the filter once would leak another upload into the prompt. A dedicated collection makes that mistake impossible.

8. Why cosine distance?

Retrieval embeddings are compared by direction, the angle between two vectors, not by raw length. Chroma's default is L2, straight-line distance. DocuRAG creates each collection with cosine space so nearest neighbor means most similar meaning.

9. What does top-k mean here?

k is how many nearest chunks are passed to the model. DocuRAG uses up to 8. Fewer chunks are more precise and easier to miss the answer. More chunks cost tokens and can distract the model. A resume is only about 8 chunks, so the cap includes the whole file and the model can connect a skill on page 1 with a project on page 2.

10. What is the stuff chain?

Stuff means concatenate the retrieved chunks into one prompt and call the model once. That is the old LangChain RetrievalQA stuff strategy. DocuRAG writes it as an LCEL chain, prompt piped into the chat model, so each step is visible. Map-reduce, summarizing every chunk and then summarizing those summaries, is only worth it when there are many more passages than fit in one prompt.

11. How do you limit hallucination?

The model never sees the raw file. It sees the question plus the retrieved excerpts. Stated facts must come from that text. Inferences, such as a salary the resume does not list, must start with "This is not stated in the document" and may use only roles, skills, dates, and locations that appear in the excerpts. The model must not invent an employer, a date, or a metric. If the excerpts are unrelated, the reply is exactly "I don't have enough information." The UI still shows the snippets, because the model can misread a passage that is in the context.

12. A resume has no salary. The user asks what they could earn. What should happen?

Do not refuse, and do not pretend the resume states a number. Retrieve the experience, skills, education, and location. Answer that pay is not stated, then give a short estimate tied to those facts, for example an entry-level full-stack range in India based on the internships and stack that are actually in the file. Label it as an estimate.

13. Where do the Sources come from?

They are the chunks Chroma returned, in rank order, best match first. Each row is filename, page if it is a PDF, chunk index, and a snippet of about 280 characters. The model is not asked to invent citations. The list is collapsed by default so the answer stays readable.

14. What is stored in the browser versus on the server?

localStorage holds the session id and the chat transcript, so a refresh does not start over. Chroma on disk holds the chunks and vectors. Clearing the session calls DELETE /documents/{session_id}, drops the collection, and wipes the local transcript.

15. Why can a scanned PDF not be indexed?

pypdf reads the text layer. A photo of a page has no text layer, so extraction returns empty and the API says the file needs OCR first. An encrypted PDF that cannot be opened is also rejected.

16. Why is the Gemini key not in the React app?

Anything in the frontend can be read in the browser. The key would be stolen and the quota spent. The React app only calls the FastAPI origin. The key is GEMINI_API_KEY on the server, loaded from the environment or a gitignored .env file.

17. Why gemini-3.8-flash instead of gemini-2.5-flash?

gemini-1.5-flash is retired. New API keys receive a 404 on gemini-2.5-flash and are told to use gemini-3.8-flash. The chat model is configurable with GEMINI_CHAT_MODEL. Temperature is 0 so answers stay close to the evidence.

18. What fails, and how does the API say so?

Unsupported type, empty file, file over 20 MB, more than 400 chunks, scanned or unreadable PDF, chatting before any chunks exist, a missing key, and a Gemini or quota error. Those are RagError values mapped to HTTP 400, 404, 500, or 502 with a JSON detail string. A failed embed rolls back the chunks just inserted so a half index is not left behind.

19. How would you take this past a laptop demo?

The shape stays the same: chunk, embed, retrieve, stuff, generate, cite. Replace local Chroma with a hosted vector database, add login so a session is a user, and add a queue for embedding so a large PDF does not block the API. Keep one index namespace per user. Do not change the embedding model without reindexing. Uvicorn stays at one worker today because Chroma's local store is a single SQLite file.

A short demo: upload a resume or samples/employee-handbook.txt. Ask a fact that is written down, such as vacation days or CGPA. Ask a judgment that is not written down, such as a likely salary, and point at the sentence that says it is not stated. Ask something unrelated and show the refusal. Open Sources and show that the snippet is the retrieved text, not a citation the model made up.